

Trabajo Práctico: Consumo de The Simpsons API con TypeScript

Objetivo

Crear una aplicación web que consuma la API de Los Simpsons para mostrar personajes de la serie con sus imágenes y frases célebres.

Requisitos previos

- Node.js instalado (versión 14 o superior)
- Editor de código (VS Code recomendado)

Parte 1: Configuración inicial del proyecto

1.1 Crear estructura del proyecto

Crear una carpeta para el proyecto y dentro generar la siguiente estructura:

```
simpsons-api-project/
|
├── src/
|   ├── styles.css
|   └── main.ts
├── index.html
├── package.json
└── tsconfig.json
```

1.2 Inicializar npm

Dentro de la carpeta del proyecto, ejecutar:

```
npm init -y
```

Consigna: Explicar en un comentario dentro del README.md qué hace este comando y qué archivo genera.

1.3 Instalar TypeScript

`npm install typescript --save-dev`

Consigna: ¿Por qué se usa `--save-dev` en lugar de guardar TypeScript como dependencia normal?

1.4 Configurar TypeScript

Crear el archivo `tsconfig.json` con la siguiente configuración:

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ES2020",
    "lib": ["ES2020", "DOM"],
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
  },
  "include": ["src/**/*.ts"],
  "exclude": ["node_modules"]
}
```

Consigna: Investigar y explicar qué hacen las opciones `strict`, `target` y `outDir`.

1.5 Configurar scripts en package.json

Agregar los siguientes scripts en `package.json`:

```
"scripts": {
  "build": "tsc",
  "watch": "tsc --watch"
}
```

Consigna: Explicar qué hace cada uno de estos scripts y cuándo se usarían.

Parte 2: Documentación de la API

The Simpsons Quote API

URL Base: `https://thesimpsonsapi.com/api/characters`

Método: GET

Ejemplo de petición:

`https://thesimpsonsapi.com/api/characters`

Respuesta de ejemplo:

```
{
  "count": 1182,
  "next": "https://thesimpsonsapi.com/api/characters?page=2",
  "prev": null,
  "pages": 60,
  "results": [
    {
      "id": 1,
      "age": 39,
      "birthdate": "1956-05-12",
      "gender": "Male",
      "name": "Homer Simpson",
      "occupation": "Safety Inspector",
      "portrait_path": "/character/1.webp",
      "phrases": ["Doh!", "Why you little...!", "Woo-hoo!"],
      "status": "Alive"
    }
  ]
}
```

Consignas:

1. Realizar peticiones de prueba a la API usando el navegador o Postman
2. Documentar la estructura de datos que devuelve la API
3. Crear una interfaz TypeScript que represente la estructura de un personaje
4. Para generar una imagen del personaje debe colocar <https://cdn.thesimpsonsapi.com/500> + el campo **portrait_path**

Parte 3: HTML

3.1 Estructura básica

Crear el archivo `index.html` con la estructura HTML5 completa que incluya:

Consignas:

1. **Header** que contenga:
 - Título principal de la aplicación
 - Subtítulo descriptivo
2. **Section de controles** que incluya:
 - Botón para "Cargar Personajes"
3. **Section de loading** (oculto por defecto):
 - Spinner o indicador de carga
 - Texto "Cargando personajes..."
4. **Div para mensajes de error** (oculto por defecto)
5. **Section para mostrar los personajes:**
 - Debe tener un ID para poder seleccionarlo desde JavaScript
 - Contendrá las tarjetas de personajes dinámicamente
6. **Footer** con:
 - Enlace a la documentación de la API
7. **Script tag:**
 - Debe importar el archivo JavaScript compilado como módulo
 - Recordar usar `type="module"`

Parte 5: TypeScript

5.1 Interfaces y tipos

Consignas:

1. Crea una interfaz para la respuesta de la api llamada `IResponseApi`
2. Crear una interfaz `SimpsonCharacter` el personaje de los simpson

5.2 Variables y constantes

2. Declarar constantes para:
 - URL base de la API
 - Elementos del DOM (boton, contenedores)
 - Usar el tipado correcto para cada elemento (`HTMLButtonElement`, `HTMLDivElement`, etc...)

5.3 Funciones requeridas

Implementar las siguientes funciones:

3. **showLoading(): void**
 - Mostrar el indicador de carga
 - Ocultar mensajes de error
4. **hideLoading(): void**
 - Ocultar el indicador de carga
5. **showError(message: string): void**
 - Mostrar mensaje de error al usuario
 - Ocultar el error automáticamente después de 5 segundos
6. **createCharacterCard(character: SimpsonCharacter): HTMLElement**
 - Crear un div con clase "character-card"
 - Agregar imagen del personaje
 - Agregar nombre del personaje (h3)
 - Agregar una frase del personaje
 - Retornar el elemento creado
7. **const renderCharacters = (characters: SimpsonCharacter[]): void**
 - Remueve los caracteres existentes
 - Recorre a los caracteres y hace uso de **createCharacterCard**
 - Hace el append al contenedor correspondiente
8. **const fetchCharacters = async (): Promise<void>**
 - Mostrar loading
 - Hacer fetch a la API
 - Verificar que la respuesta sea exitosa (response.ok)
 - Parsear JSON
 - Hace uso de **renderCharacters**
 - Ocultar loading
 - Manejar errores con try/catch
 - Mostrar mensaje de error si algo falla

5.4 Event Listeners

10. Agregar event listeners para:
 - Click en botón "Cargar Personajes" → llamar fetchCharacters con el valor del input

5.5 Manejo de errores

11. Implementar manejo de errores robusto:
 - Try/catch en funciones async
 - Validación de datos
 - Mensajes de error descriptivos
 - Console.error para debugging

Parte 6: Ejecución y pruebas

Comandos para ejecutar el proyecto:

1. Compilar TypeScript:

npm run build

2. Compilar en modo watch (recompila automáticamente):

npm run watch

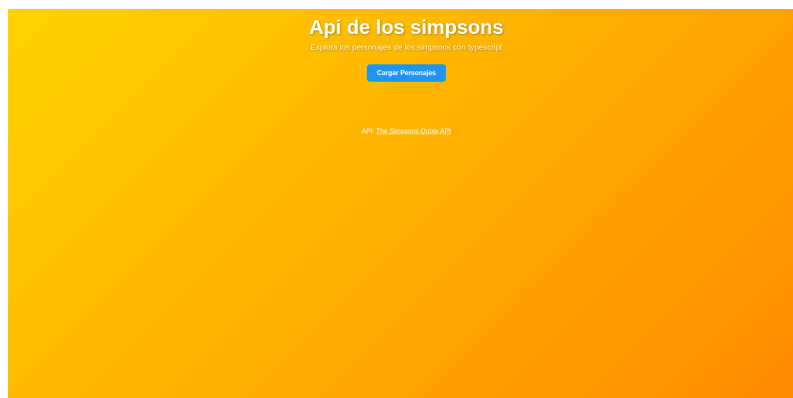
3. Iniciar servidor:

Consignas:

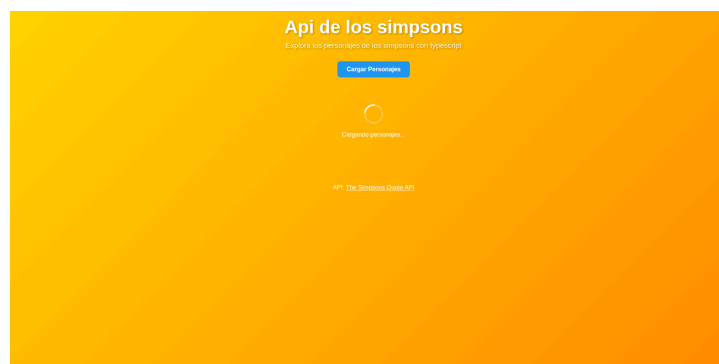
- Probar todas las funcionalidades
- Verificar que no haya errores en la consola
- Probar diferentes cantidades de personajes
- Verificar responsive design en diferentes dispositivos

Vista de ejemplo

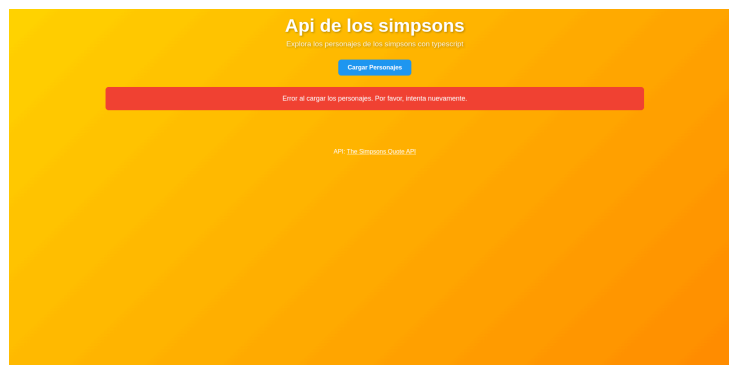
Vista a penas se abre la app



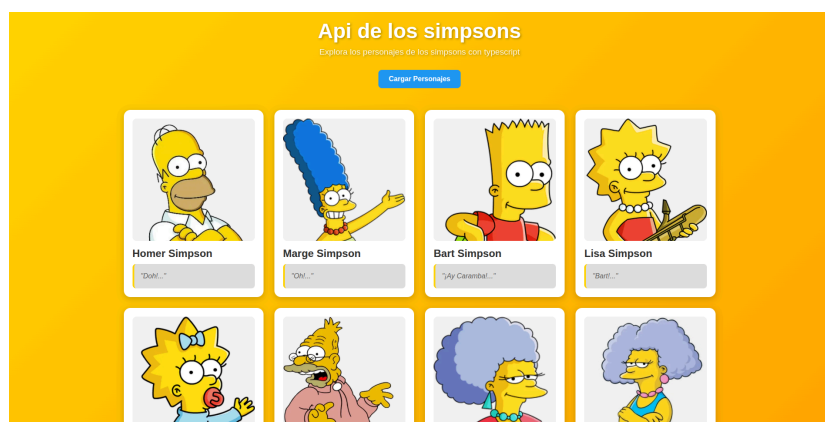
Vista con loading



Vista con error



vista con los personajes



¡Éxito con el trabajo práctico! 🍕